

Lab 13 (CS): Deadlocks

Innopolis University
Course of Operating Systems

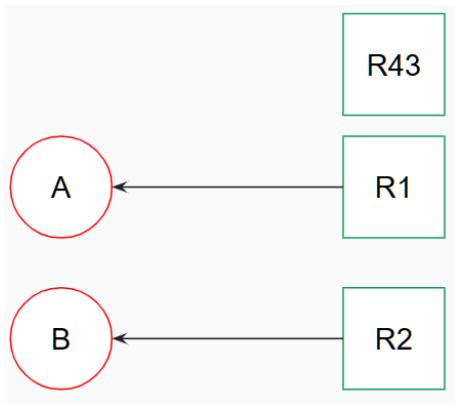
Deadlock

- Resource
 - An object granted is called a resource. (Hardware or Software)
 - A resource can be a hardware device (e.g., a Blu-ray drive) or a piece of information (e.g., a record in a database).
- A **preemptable** resource
 - can be taken away from the process owning it with no ill effects.
 - The reallocation of the same resource must have no negative effect on the processes involved.
 - Memory, File.
- A **non-preemptable** resource
 - cannot be taken away from its current owner without potentially causing failure
 - Printer.
 - Source of deadlocks
- How to use the resource.
 - Request the resource
 - Use the resource
 - Release the resource

Deadlock

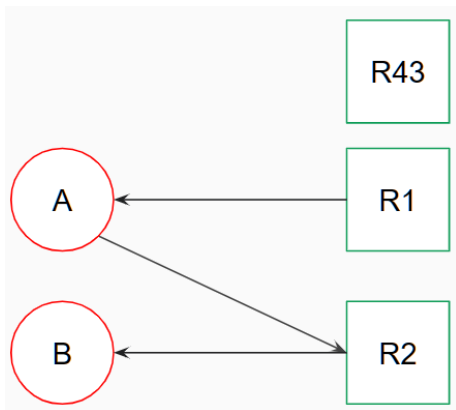
- A set of blocked processes each holding a resource and waiting to acquire a resource held by another process in the set.
- **Coffman Conditions:**
 - *Mutual Exclusion Condition:* Only one process can use the resource at any given instant of time.
 - *Hold-and-Wait Condition:* a process is currently holding at least one resource and requesting additional resources which are being held by other processes.
 - *No preemption Condition:* a resource can be released only voluntarily by the process holding it.
 - *Circular wait Condition:* each process must be waiting for a resource which is being held by another process, which in turn is waiting for the first process to release the resource.
- A deadlock situation on a resource can arise if and only if all of the **Coffman conditions** occur simultaneously in a system.

Coffman conditions



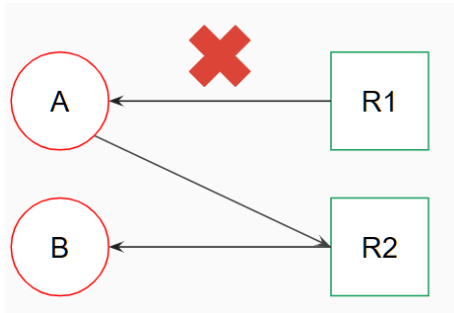
Which conditions are satisfied here?

Coffman conditions



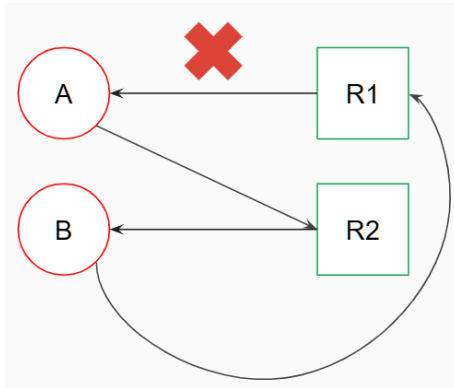
Which conditions are satisfied here?

Coffman conditions



Which conditions are satisfied here?

Coffman conditions



Which conditions are satisfied here?

Dealing with Deadlock

- Ignore (Ostrich Approach)
 - Don't detect and don't handle the deadlock
- Detect and Recover
 - Detect and handle the deadlock
- Avoid
 - Carefully, allocate the resources. Is it 100% guaranteed?
- Prevent
 - By reviewing Coffman conditions Try to break at least one condition to prevent the deadlock.

Deadlock detection algorithm (one Resource of each type)

- 1. For each node N in the graph, perform the following 5 steps with N as the starting node
- 2. Initialize L (i.e., list of nodes) to the empty list and designate all the arcs as unmarked
- 3. Add the current node to the end of L and check to see if the node now appears in L two times. If it does, the graph contains a cycle (listed in L) and the algorithm terminates
- 4. For the given node, see if there are any unmarked outgoing arcs. If so, go to step 5; if not, go to step 6.
- 5. Pick an unmarked outgoing arc at random and mark it. Then follow it to the new current node and go to step 3.
- 6. We have now reached a dead end. Remove it and go back to the previous node, that is, the one that was current just before this one, make that one the current node, and go to step 3. IF this node is the initial node, the graph does not contain any cycles and the algorithm terminates.

Deadlock detection algorithm (one Resource of each type)

- The question is: is this system deadlocked, and if so, which processes are involved?
- Let's construct a resource graph.

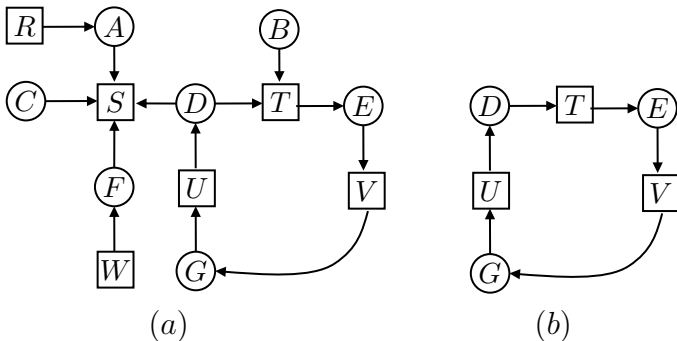


Figure 6-5. (a) A resource graph, (b) A cycle extracted from (a)

Exercise 1

- Write a C program `ex1.c` which creates `n` threads and `m` resource types which are protected by mutexes. It processes a list of resource requests read from an input file `input.txt`. Each request needs to specify the ID of the resource type, and the thread ID. The program should read `m` and `n` as command line arguments (`./ex1 $m $n`).
- The input format is as follows:

thread resource

1 2

3 0

1 0

3 2

1 1

3 1

Exercise 1

- The program should process the request (`thread=i`, `resource=j`) as follows:
 1. Create the thread `i` if not existed
 2. Print a message that thread `i` is created
 3. Print the ids of mutexes currently locked by the thread `i`
 4. Print a message that thread `i` tries to lock the mutex `j`
 5. If it cannot lock, then the program should check if there is a deadlock
 6. The thread `i` locks the mutex `j` for a random number of seconds `x` (`sleep(x)`) where $0 < x < 5$
 7. The thread `i` unlocks the mutex `j`
 8. Print a message that thread `i` unlocked mutex `j`
 9. If all acquired mutexes are unlocked by thread `i` then terminate it before returning from `start_routine` function

Exercise 1

- When a deadlock happens, the program should print a message that “Deadlock is detected” and terminate after printing the status of all threads (which mutexes are locked and requested by which threads).
- If the program finishes processing all requests and no deadlocks occurred then the program should print a message that “No deadlocks”.
- The program should process requests in parallel. After a thread is created, the program should not wait till terminating the current thread (using join) but indeed it should process the next request (creating another thread). You can join threads after finishing all requests.
- If the thread i locked a mutex then any threads cannot lock the same mutex before unlocking by thread i . We assume that we have only one instance of each resource type (protected by a mutex).

Exercise 1

- You should solve the problem based on the algorithm presented in slide 9. You can use any data structure or representation you prefer in C to store the nodes and edges of the graph.
- Save the output of your program in a file **ex1.txt** and add to it the answer to the question. Why did the deadlock occur if it is a deadlock? and Why it did not happen if it is not a deadlock?
- Submit **ex1.c**, **ex1.txt** and script **ex1.sh** to run the program.

- Let the number of resource classes be m , with E_1 resources of class 1, E_2 resources of class 2, and generally, E_i resources of class i ($1 \leq i \leq m$).
- E is the existing resource vector. It gives the total number of instances of each resource in existence
- Let A be the available resource vector, with A_i equals the number of instances of resource i that are currently available (i.e., unassigned)

- We have two arrays:
 - C, the current allocation matrix
 - R, the request matrix

Resources in existence
 $(E_1, E_2, E_3, \dots, E_m)$

Resources available
 $(A_1, A_2, A_3, \dots, A_m)$

Current allocation matrix

$$\begin{bmatrix} C_{11} & C_{12} & C_{13} & \dots & C_{1m} \\ C_{21} & C_{22} & C_{23} & \dots & C_{2m} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ C_{n1} & C_{n2} & C_{n3} & \dots & C_{nm} \end{bmatrix}$$

Row n is current
 allocation
 to process n

Request matrix

$$\begin{bmatrix} R_{11} & R_{12} & R_{13} & \dots & R_{1m} \\ R_{21} & R_{22} & R_{23} & \dots & R_{2m} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ R_{n1} & R_{n2} & R_{n3} & \dots & R_{nm} \end{bmatrix}$$

Row 2 is what process 2
 needs

Deadlock Avoidance (1/2)

Banker's Algorithm:

- The algorithm checks if granting the request leads to an unsafe state.
- In that case, the request is denied.

Deadlock Avoidance (2/2)

The algorithm for checking to see if a state is safe can be stated as:

- Look for a row, R , whose unmet resource needs are all smaller than or equal to A . If no such row exists, the system will eventually be deadlocked since no process can run to completion.
- Assume the process of the chosen row requests all the resources it needs (which is guaranteed to be possible) and finishes. Mark that process as terminated and add all of its resources to the A vector.
- Repeat steps 1 and 2 until either all processes are marked terminated (in which case the initial state was safe) or no process is left whose resource needs can be met (in which case the system was not safe).

Exercise 2 (1/2) (1/2 bonus points)

- Write a C program **ex2.c** to implement the Banker's Algorithm for deadlock avoidance:
 - Program should read the input from a file (input.txt)
 - For testing purposes consider 5 processes and 3 types of resources. However, your program must be able to process as many processes and resource types, as needed (check next slide for input file structure description)
 - Your program should either say that no deadlock is detected or print out the processes that are deadlocked.
 - Submit **ex2.c**.

Exercise 2 (2/2) (1/2 bonus points)

- Data shown on the left will be stored in the input file as shown on the right

```
- E = (7 2 6)
- A = (0 0 0)
- Current allocation matrix:
C =
0 1 0
2 0 0
3 0 3
2 1 1
0 0 2
- Request Matrix
R =
0 0 0
2 0 2
0 0 0
1 0 0
0 0 2
```

Data

```
7 2 6
0 0 0
0 1 0
2 0 0
3 0 3
2 1 1
0 0 2
0 0 0
2 0 2
0 0 0
1 0 0
0 0 2
```

Input file

High-level Pseudo code of Banker's algorithm

- Read R and C matrices, A and E vectors from input.txt file.
- All processes are **unmarked** by default.
- Function for checking the resource request of the process i .
 - True - All elements of row i in R matrix $\geq$ elements of vector A.
 - The request of process i can be fulfilled.
 - False - Otherwise.
- While (there are resource requests that **can** be fulfilled)
 - For each process i
 - If the process i is unmarked and its request can be fulfilled.
 - Mark the process.
 - Update vector **A** by releasing all resources of process i from **C**.
- If still there are Unmarked processes
 - **Deadlock**
- The unmarked processes are those deadlocked.

End of lab 13 (CS)